

```
import numpy as np
import pandas as pd
import nltk
nltk.download('punkt_tab')
nltk.download('stopwords')
import string
from nltk.corpus import stopwords
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
```

```
df = pd.read_csv('/content/spam.csv',encoding='latin-1')
```

```
df.sample(5)
```

```
↗
```

	v1	v2	Unnamed: 2	Unnamed: 3	Unnamed: 4
4090	ham	Hey doc pls I want to get nice t shirt for my ...	NaN	NaN	NaN
4042	ham	I am literally in bed and have been up for lik...	NaN	NaN	NaN
3763	ham	Would me smoking you out help us work through ...	NaN	NaN	NaN
4239	ham	The LAY MAN! Just to let you know you are miss...	NaN	NaN	NaN
2474	ham	Princess, i like to make love & time...	NaN	NaN	NaN

```
df.shape
```

```
↗ (5572, 5)
```

```
# 1. Data cleaning
# 2. EDA
# 3. Text Preprocessing
# 4. Model building
# 5. Evaluation
# 6. Improvement
# 7. Website
# 8. Deploy
```

✓ 1. Data Cleaning

```
df.info()
```

```
↗ <class 'pandas.core.frame.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0    v1          5572 non-null   object
1    v2          5572 non-null   object
2    Unnamed: 2   50 non-null     object
3    Unnamed: 3   12 non-null     object
4    Unnamed: 4    6 non-null      object
dtypes: object(5)
memory usage: 217.8+ KB
```

```
# drop last 3 cols
df.drop(columns=['Unnamed: 2','Unnamed: 3','Unnamed: 4'],inplace=True)
```

```
df.sample(5)
```

	v1	v2
4454	ham	Storming msg: Wen u lift d phne, u say \HELLO\...
2892	ham	K I'll be sure to get up before noon and see w...
4712	ham	Big brother's really scraped the barrel with...
1049	spam	18 days to Euro2004 kickoff! U will be kept in...
2620	ham	How come?

```
# renaming the cols
df.rename(columns={'v1': 'target', 'v2': 'text'}, inplace=True)
df.sample(5)
```

	target	text
233	ham	Yes:)here tv is always available in work place..
4256	spam	important information 4 orange user . today is...
188	ham	Have you got Xmas radio times. If not i will g...
599	ham	I'm going out to buy mum's present ar.
5113	ham	U definitely need a module from e humanities d...

```
from sklearn.preprocessing import LabelEncoder
encoder = LabelEncoder()
```

```
df['target'] = encoder.fit_transform(df['target'])
```

```
df.head()
```

	target	text
0	0	Go until jurong point, crazy.. Available only ...
1	0	Ok lar... Joking wif u oni...
2	1	Free entry in 2 a wkly comp to win FA Cup fina...
3	0	U dun say so early hor... U c already then say...
4	0	Nah I don't think he goes to usf, he lives aro...

Next steps:

[Generate code with df](#)
[View recommended plots](#)
[New interactive sheet](#)

```
# missing values
df.isnull().sum()
```

```
0
target 0
text 0
dtype: int64
```

```
# check for duplicate values
df.duplicated().sum()
```

```
np.int64(403)
```

```
# remove duplicates
df = df.drop_duplicates(keep='first')
```

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df.shape
```

 (5169, 2)

2.EDA

```
df.head()
```

	target	text
0	0	Go until jurong point, crazy.. Available only ...
1	0	Ok lar... Joking wif u oni...
2	1	Free entry in 2 a wkly comp to win FA Cup fina...
3	0	U dun say so early hor... U c already then say...
4	0	Nah I don't think he goes to usf, he lives aro...

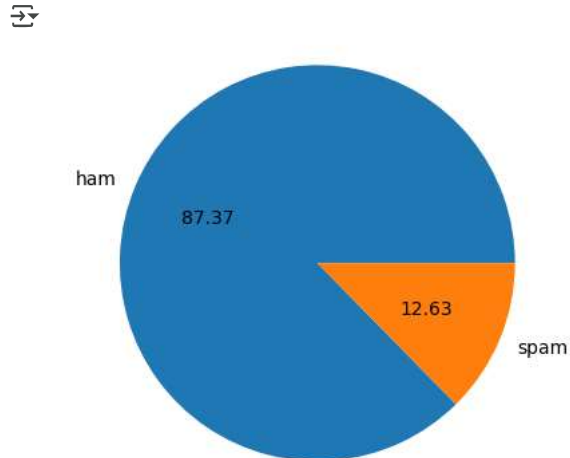
Next steps: [Generate code with df](#) [View recommended plots](#) [New interactive sheet](#)

```
df['target'].value_counts()
```

	count
target	
0	4516
1	653

dtype: int64

```
import matplotlib.pyplot as plt
plt.pie(df['target'].value_counts(), labels=['ham', 'spam'], autopct="%0.2f")
plt.show()
```



Data is imbalanced

```
import nltk
```

```
!pip install nltk
```

```
 Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.2.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.1)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2024.11.6)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.1)
```

```
nltk.download('punkt')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
True
```

```
df['num_characters'] = df['text'].apply(len)
```

```
df.head()
```



	target	text	num_characters	
0	0	Go until jurong point, crazy.. Available only ...	111	
1	0	Ok lar... Joking wif u oni...	29	
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155	
3	0	U dun say so early hor... U c already then say...	49	
4	0	Nah I don't think he goes to usf, he lives aro...	61	

Next steps:

[Generate code with df](#)

[View recommended plots](#)

[New interactive sheet](#)

```
# num of words
df['num_words'] = df['text'].apply(lambda x:len(nltk.word_tokenize(x)))
```

```
df.head()
```



	target	text	num_characters	num_words	
0	0	Go until jurong point, crazy.. Available only ...	111	24	
1	0	Ok lar... Joking wif u oni...	29	8	
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155	37	
3	0	U dun say so early hor... U c already then say...	49	13	
4	0	Nah I don't think he goes to usf, he lives aro...	61	15	

Next steps:

[Generate code with df](#)

[View recommended plots](#)

[New interactive sheet](#)

```
df['num_sentences'] = df['text'].apply(lambda x:len(nltk.sent_tokenize(x)))
```

```
df.head()
```



	target	text	num_characters	num_words	num_sentences	
0	0	Go until jurong point, crazy.. Available only ...	111	24	2	
1	0	Ok lar... Joking wif u oni...	29	8	2	
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155	37	2	
3	0	U dun say so early hor... U c already then say...	49	13	1	
4	0	Nah I don't think he goes to usf, he lives aro...	61	15	1	

Next steps:

[Generate code with df](#)

[View recommended plots](#)

[New interactive sheet](#)

```
df[['num_characters', 'num_words', 'num_sentences']].describe()
```



	num_characters	num_words	num_sentences
count	5169.000000	5169.000000	5169.000000
mean	78.977945	18.455794	1.965564
std	58.236293	13.324758	1.448541
min	2.000000	1.000000	1.000000
25%	36.000000	9.000000	1.000000
50%	60.000000	15.000000	1.000000
75%	117.000000	26.000000	2.000000
max	910.000000	220.000000	38.000000




```
# ham
df[df['target'] == 0][['num_characters', 'num_words', 'num_sentences']].describe()
```



	num_characters	num_words	num_sentences
count	4516.000000	4516.000000	4516.000000
mean	70.459256	17.123782	1.820195
std	56.358207	13.493970	1.383657
min	2.000000	1.000000	1.000000
25%	34.000000	8.000000	1.000000
50%	52.000000	13.000000	1.000000
75%	90.000000	22.000000	2.000000
max	910.000000	220.000000	38.000000




```
#spam
df[df['target'] == 1][['num_characters', 'num_words', 'num_sentences']].describe()
```



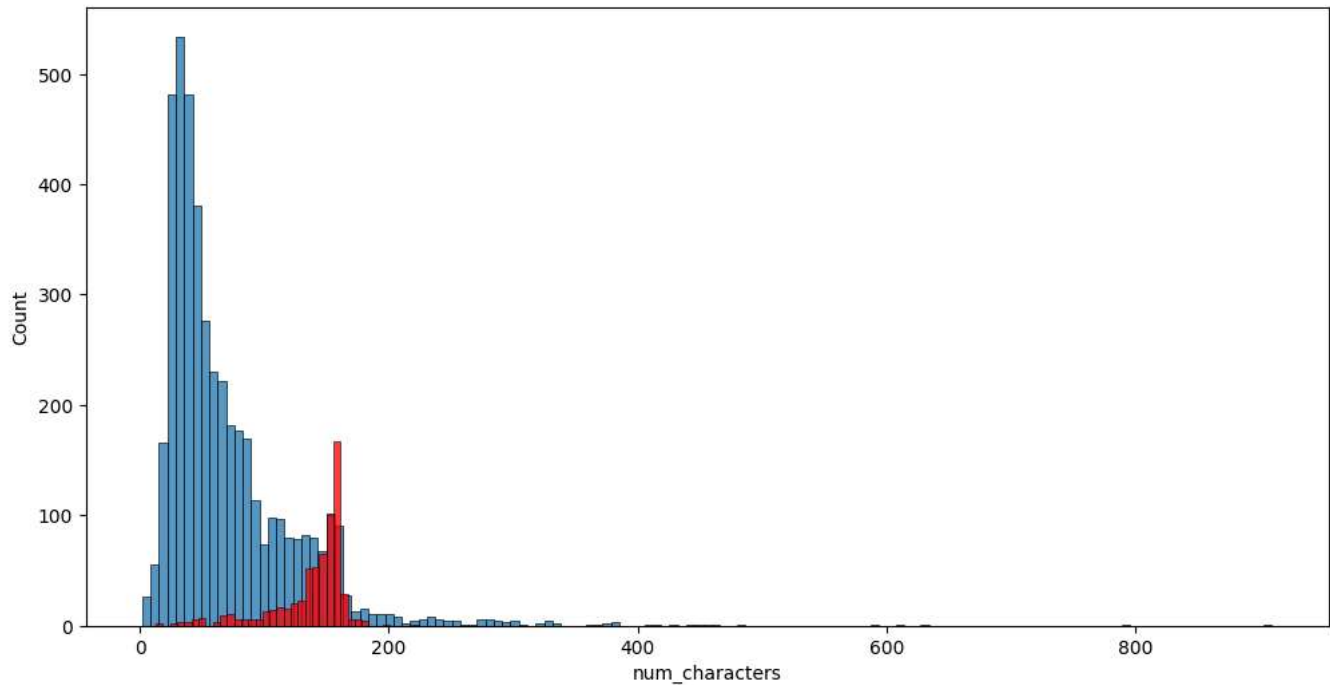
	num_characters	num_words	num_sentences
count	653.000000	653.000000	653.000000
mean	137.891271	27.667688	2.970904
std	30.137753	7.008418	1.488425
min	13.000000	2.000000	1.000000
25%	132.000000	25.000000	2.000000
50%	149.000000	29.000000	3.000000
75%	157.000000	32.000000	4.000000
max	224.000000	46.000000	9.000000




```
import seaborn as sns
```

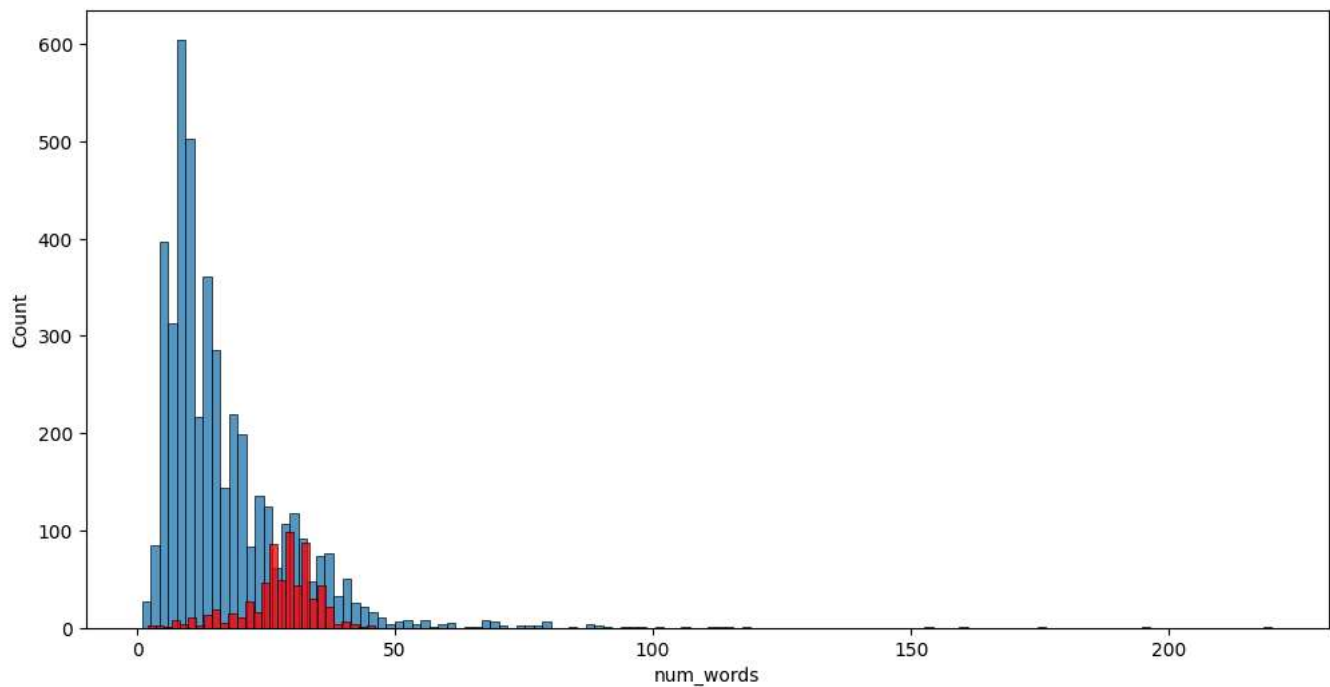
```
plt.figure(figsize=(12,6))
sns.histplot(df[df['target'] == 0]['num_characters'])
sns.histplot(df[df['target'] == 1]['num_characters'],color='red')
```

 <Axes: xlabel='num_characters', ylabel='Count'>



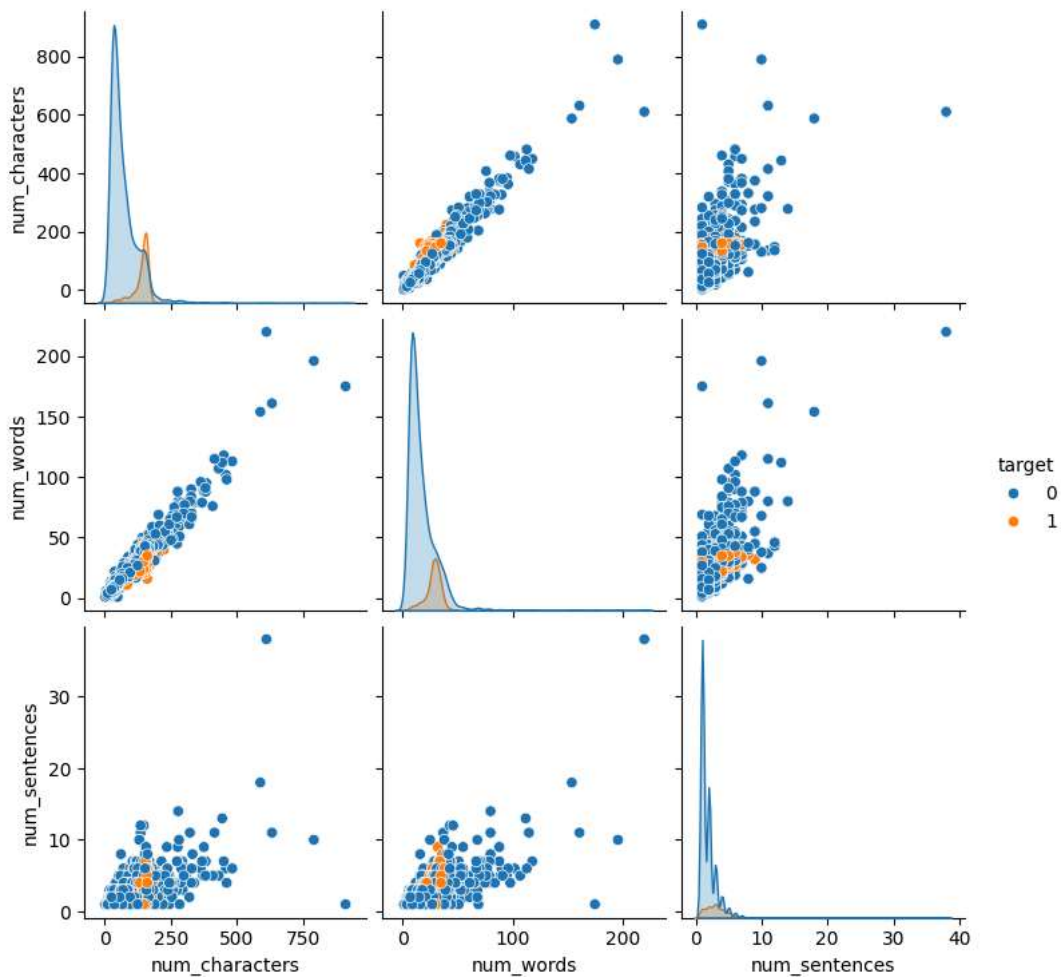
```
plt.figure(figsize=(12,6))
sns.histplot(df[df['target'] == 0]['num_words'])
sns.histplot(df[df['target'] == 1]['num_words'],color='red')
```

 <Axes: xlabel='num_words', ylabel='Count'>



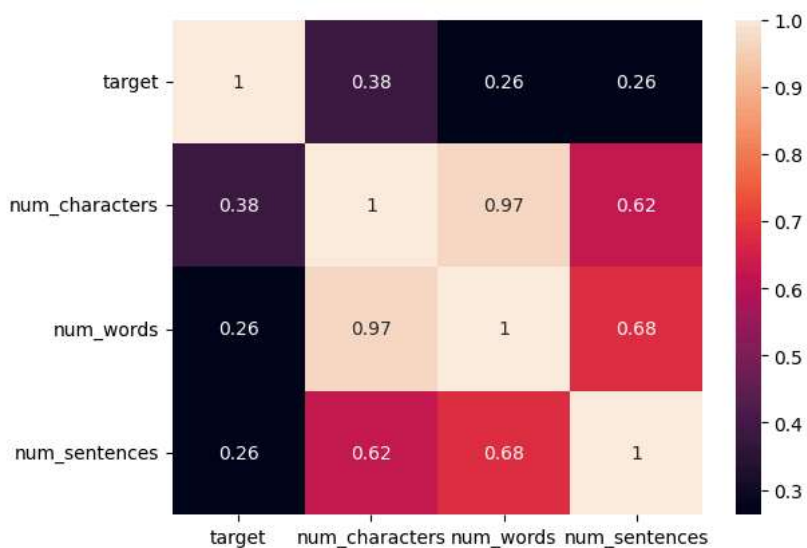
```
sns.pairplot(df,hue='target')
```

 <seaborn.axisgrid.PairGrid at 0x7b7b19221220>



```
sns.heatmap(df.corr(numeric_only=True),annot=True)
```

 <Axes: >



3. Data Preprocessing

- Lower case
- Tokenization
- Removing special characters
- Removing stop words and punctuation

- Stemming

```
from nltk.stem.porter import PorterStemmer

def transform_text(text):
    text = text.lower()
    text = nltk.word_tokenize(text)

    y = []
    for i in text:
        if i.isalnum():
            y.append(i)

    text = y[:]
    y.clear()

    for i in text:
        if i not in stopwords.words('english') and i not in string.punctuation:
            y.append(i)

    text = y[:]
    y.clear()

    ps = PorterStemmer()
    for i in text:
        y.append(ps.stem(i))

    return " ".join(y)

transform_text("I'm gonna be home soon and i don't want to talk about this stuff anymore tonight, k? I've cried enough today.")
```

 'gon na home soon want talk stuff anymor tonight k cri enough today'

```
df['text'][10]

 'I'm gonna be home soon and i don't want to talk about this stuff anymore tonight, k? I've cried enough today.'
```

```
from nltk.stem.porter import PorterStemmer
ps = PorterStemmer()
ps.stem('loving')

 'love'
```

```
df['transformed_text'] = df['text'].apply(transform_text)
```

```
df.head()
```



	target	text	num_characters	num_words	num_sentences	transformed_text
0	0	Go until jurong point, crazy.. Available only ...	111	24	2	go jurong point crazi avail bugi n great world...
1	0	Ok lar... Joking wif u oni...	29	8	2	ok lar joke wif u oni
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155	37	2	free entri 2 wkli comp win fa cup final tkt 21...
3	0	U dun say so early hor... U c already then say...	49	13	1	u dun say earli hor u c already say

Next steps: [Generate code with df](#) [View recommended plots](#) [New interactive sheet](#)

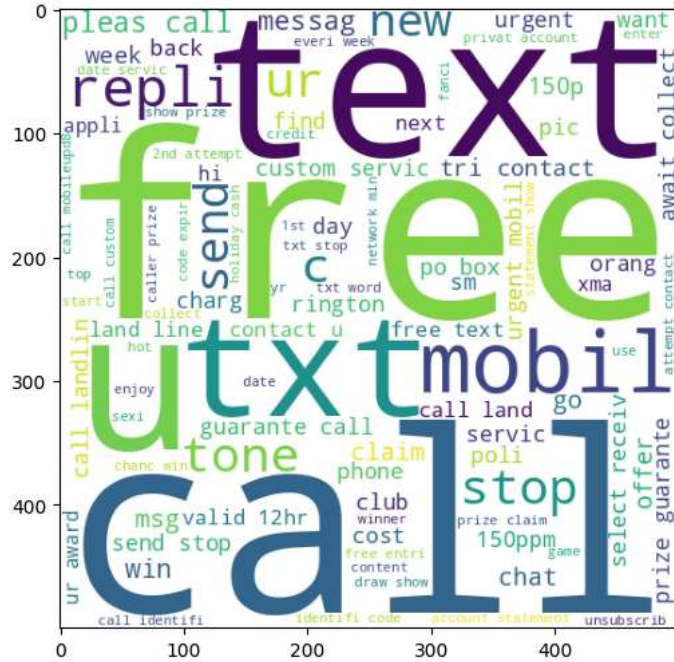
```
from wordcloud import WordCloud
wc = WordCloud(width=500,height=500,min_font_size=10,background_color='white')

spam_wc = wc.generate(df[df['target'] == 1]['transformed_text'].str.cat(sep=" "))

plt.figure(figsize=(15,6))
plt.imshow(spam_wc)
```



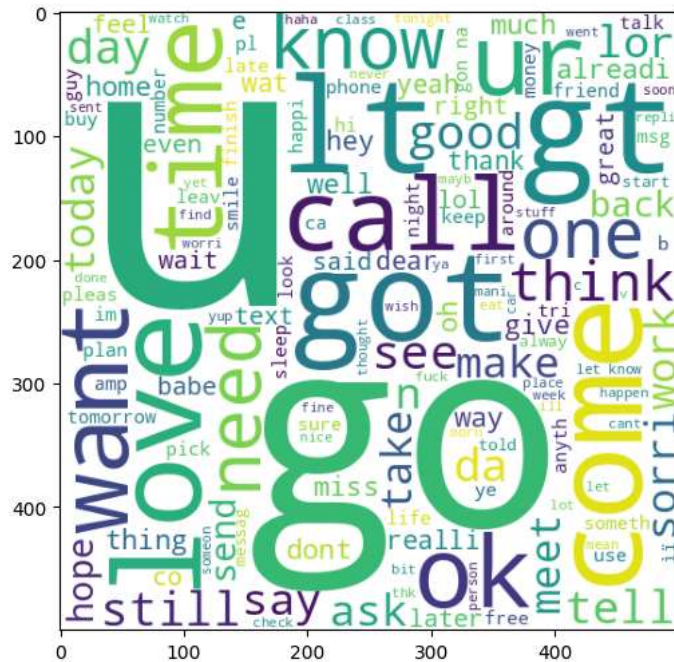
```
→ <matplotlib.image.AxesImage at 0x7b7b18c3ad20>
```



```
ham_wc = wc.generate(df[df['target'] == 0]['transformed_text'].str.cat(sep=" "))
```

```
plt.figure(figsize=(15,6))
plt.imshow(ham_wc)
```

```
→ <matplotlib.image.AxesImage at 0x7b7b18888c20>
```



```
df.head()
```

Text processing							
	target	text	num_characters	num_words	num_sentences	transformed_text	
0	0	Go until jurong point, crazy.. Available only ...	111	24	2	go jurong point crazi avail bugi n great world...	
1	0	Ok lar... Joking wif u oni...	29	8	2	ok lar joke wif u oni	
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155	37	2	free entri 2 wkli comp win fa cup final tkt 21...	
3	0	U dun say so early hor... U c already then say...	49	13	1	u dun say earli hor u c already say	

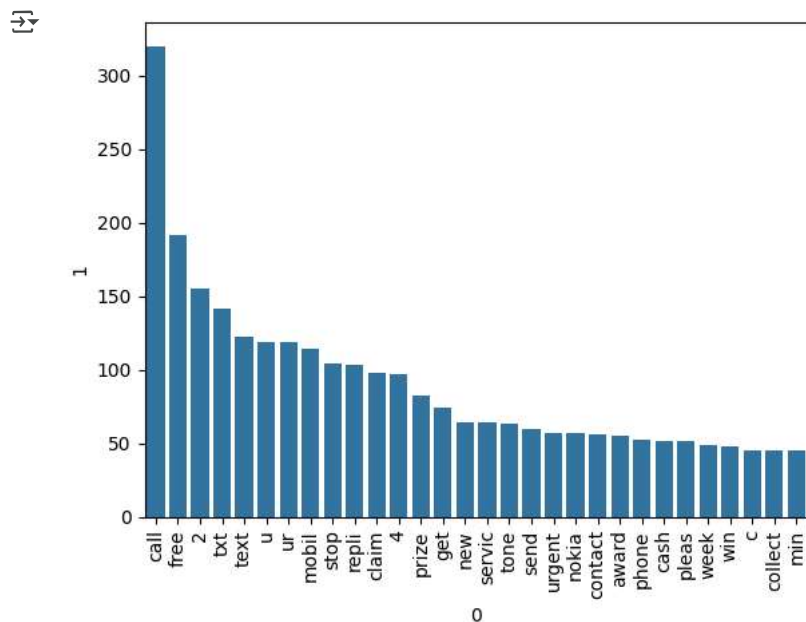
Next steps: [Generate code with df](#) [View recommended plots](#) [New interactive sheet](#)

```
spam_corpus = []
for msg in df[df['target'] == 1]['transformed_text'].tolist():
    for word in msg.split():
        spam_corpus.append(word)
```

```
len(spam_corpus)
```

9939

```
from collections import Counter
sns.barplot(x=pd.DataFrame(Counter(spam_corpus).most_common(30))[0],y=pd.DataFrame(Counter(spam_corpus).most_common(30))[1])
plt.xticks(rotation='vertical')
plt.show()
```

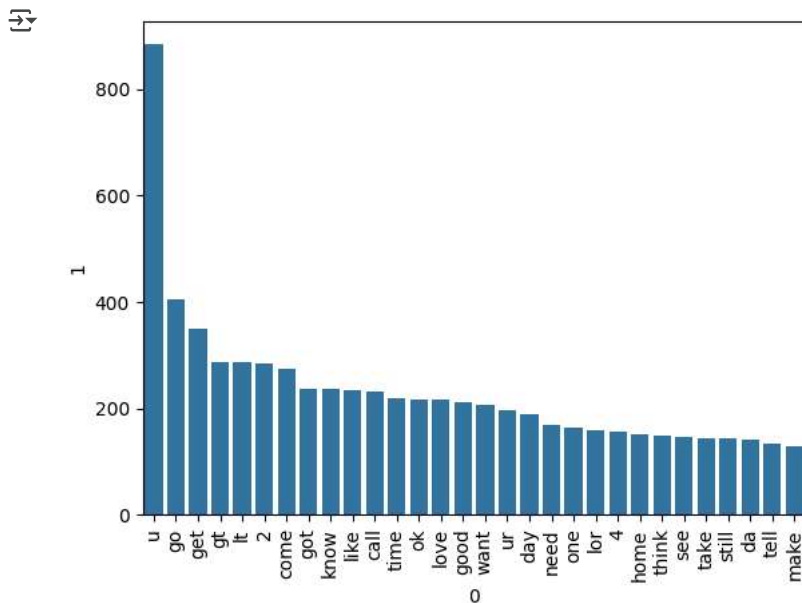


```
ham_corpus = []
for msg in df[df['target'] == 0]['transformed_text'].tolist():
    for word in msg.split():
        ham_corpus.append(word)
```

```
len(ham_corpus)
```

35404

```
from collections import Counter
sns.barplot(x=pd.DataFrame(Counter(ham_corpus).most_common(30))[0],y=pd.DataFrame(Counter(ham_corpus).most_common(30))[1])
plt.xticks(rotation='vertical')
plt.show()
```



```
# Text Vectorization
# using Bag of Words
df.head()
```

	target	text	num_characters	num_words	num_sentences	transformed_text
0	0	Go until jurong point, crazy.. Available only ...	111	24	2	go jurong point crazi avail bugi n great world...
1	0	Ok lar... Joking wif u oni...	29	8	2	ok lar joke wif u oni
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155	37	2	free entri 2 wkli comp win fa cup final tkt 21...
3	0	U dun say so early hor... U c already then say...	49	13	1	u dun say earli hor u c already say

Next steps: [Generate code with df](#) [View recommended plots](#) [New interactive sheet](#)

4. Model Building

```
from sklearn.feature_extraction.text import CountVectorizer,TfidfVectorizer
cv = CountVectorizer()
tfidf = TfidfVectorizer(max_features=3000)
```

```
X = tfidf.fit_transform(df['transformed_text']).toarray()
```

```
#from sklearn.preprocessing import MinMaxScaler
#scaler = MinMaxScaler()
#X = scaler.fit_transform(X)
```

```
# appending the num_character col to X
#X = np.hstack((X,df['num_characters'].values.reshape(-1,1)))
```

```
X.shape
```

```
(5169, 3000)
```

```
y = df['target'].values
```

```
from sklearn.model_selection import train_test_split
```

```
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=2)
```

```
from sklearn.naive_bayes import GaussianNB,MultinomialNB,BernoulliNB
from sklearn.metrics import accuracy_score,confusion_matrix,precision_score
```

```
gnb = GaussianNB()
mnb = MultinomialNB()
bnb = BernoulliNB()
```

```
gnb.fit(X_train,y_train)
y_pred1 = gnb.predict(X_test)
print(accuracy_score(y_test,y_pred1))
print(confusion_matrix(y_test,y_pred1))
print(precision_score(y_test,y_pred1))
```

```
0.874274661508704
[[790 106]
 [ 24 114]]
0.5181818181818182
```

```
mnb.fit(X_train,y_train)
y_pred2 = mnb.predict(X_test)
print(accuracy_score(y_test,y_pred2))
print(confusion_matrix(y_test,y_pred2))
print(precision_score(y_test,y_pred2))
```

```
0.9709864603481625
[[896   0]
 [ 30 108]]
1.0
```

```
bnb.fit(X_train,y_train)
y_pred3 = bnb.predict(X_test)
print(accuracy_score(y_test,y_pred3))
print(confusion_matrix(y_test,y_pred3))
print(precision_score(y_test,y_pred3))
```

```
0.9835589941972921
[[895   1]
 [ 16 122]]
0.991869918699187
```

```
# tfidf --> MNB
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.naive_bayes import MultinomialNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.ensemble import BaggingClassifier
from sklearn.ensemble import ExtraTreesClassifier
from sklearn.ensemble import GradientBoostingClassifier
from xgboost import XGBClassifier
```

```
svc = SVC(kernel='sigmoid', gamma=1.0)
knc = KNeighborsClassifier()
mnb = MultinomialNB()
dtc = DecisionTreeClassifier(max_depth=5)
lrc = LogisticRegression(solver='liblinear', penalty='l1')
rfc = RandomForestClassifier(n_estimators=50, random_state=2)
abc = AdaBoostClassifier(n_estimators=50, random_state=2)
bc = BaggingClassifier(n_estimators=50, random_state=2)
etc = ExtraTreesClassifier(n_estimators=50, random_state=2)
gbdt = GradientBoostingClassifier(n_estimators=50, random_state=2)
xgb = XGBClassifier(n_estimators=50, random_state=2)
```

```
clfs = {
    'SVC' : svc,
    'KN' : knc,
    'NB': mnb,
    'DT': dtc,
    'LR': lrc,
    'RF': rfc,
```

```

'AdaBoost': abc,
'BgC': bc,
'ETC': etc,
'GBDT': gbd,
'xgb': xgb
}

```

```

def train_classifier(clf,X_train,y_train,X_test,y_test):
    clf.fit(X_train,y_train)
    y_pred = clf.predict(X_test)
    accuracy = accuracy_score(y_test,y_pred)
    precision = precision_score(y_test,y_pred)

```

```

    return accuracy,precision

```

```

train_classifier(svc,X_train,y_train,X_test,y_test)

```

```

➡ (0.9758220502901354, 0.9747899159663865)

```

```

accuracy_scores = []
precision_scores = []

```

```

for name,clf in clfs.items():

```

```

    current_accuracy,current_precision = train_classifier(clf, X_train,y_train,X_test,y_test)

```

```

    print("For ",name)
    print("Accuracy - ",current_accuracy)
    print("Precision - ",current_precision)

```

```

    accuracy_scores.append(current_accuracy)
    precision_scores.append(current_precision)

```

```

➡ For SVC
Accuracy - 0.9758220502901354
Precision - 0.9747899159663865
For KN
Accuracy - 0.9052224371373307
Precision - 1.0
For NB
Accuracy - 0.9709864603481625
Precision - 1.0
For DT
Accuracy - 0.9303675048355899
Precision - 0.83
For LR
Accuracy - 0.9555125725338491
Precision - 0.96
For RF
Accuracy - 0.9738878143133463
Precision - 0.9826086956521739
For AdaBoost
Accuracy - 0.9216634429400387
Precision - 0.8202247191011236
For BgC
Accuracy - 0.9584139264990329
Precision - 0.8682170542635659
For ETC
Accuracy - 0.9748549323017408
Precision - 0.9745762711864406
For GBDT
Accuracy - 0.9506769825918762
Precision - 0.9306930693069307
For xgb
Accuracy - 0.9680851063829787
Precision - 0.9487179487179487

```

```

performance_df = pd.DataFrame({'Algorithm':clfs.keys(),'Accuracy':accuracy_scores,'Precision':precision_scores}).sort_values('Precision',asc

```

```

performance_df

```

	Algorithm	Accuracy	Precision	
1	KN	0.905222	1.000000	
2	NB	0.970986	1.000000	
5	RF	0.973888	0.982609	
0	SVC	0.975822	0.974790	
8	ETC	0.974855	0.974576	
4	LR	0.955513	0.960000	
10	xgb	0.968085	0.948718	
9	GBDT	0.950677	0.930693	
7	BgC	0.958414	0.868217	
3	DT	0.930368	0.830000	
6	AdaBoost	0.921663	0.820225	

Next steps:

[Generate code with performance_df](#)[View recommended plots](#)[New interactive sheet](#)

```
performance_df1 = pd.melt(performance_df, id_vars = "Algorithm")
```

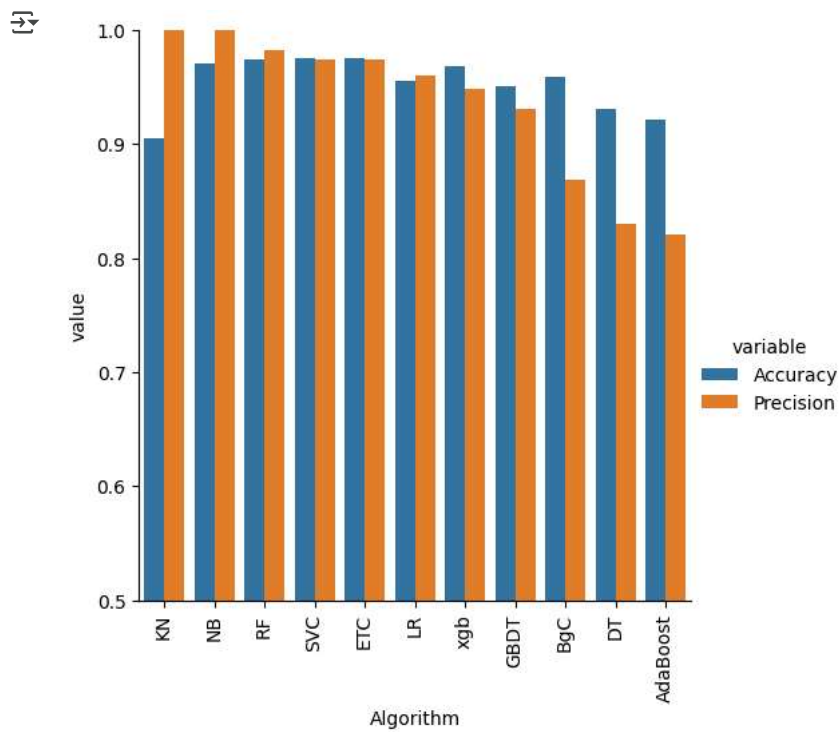
performance_df1

	Algorithm	variable	value	
0	KN	Accuracy	0.905222	
1	NB	Accuracy	0.970986	
2	RF	Accuracy	0.973888	
3	SVC	Accuracy	0.975822	
4	ETC	Accuracy	0.974855	
5	LR	Accuracy	0.955513	
6	xgb	Accuracy	0.968085	
7	GBDT	Accuracy	0.950677	
8	BgC	Accuracy	0.958414	
9	DT	Accuracy	0.930368	
10	AdaBoost	Accuracy	0.921663	
11	KN	Precision	1.000000	
12	NB	Precision	1.000000	
13	RF	Precision	0.982609	
14	SVC	Precision	0.974790	
15	ETC	Precision	0.974576	
16	LR	Precision	0.960000	
17	xgb	Precision	0.948718	
18	GBDT	Precision	0.930693	
19	BgC	Precision	0.868217	
20	DT	Precision	0.830000	
21	AdaBoost	Precision	0.820225	

Next steps:

[Generate code with performance_df1](#)[View recommended plots](#)[New interactive sheet](#)

```
sns.catplot(x = 'Algorithm', y='value',
            hue = 'variable',data=performance_df1, kind='bar',height=5)
plt.ylim(0.5,1.0)
plt.xticks(rotation='vertical')
plt.show()
```



```
# model improve
```

```
# 1. Change the max_features parameter of Tfidf
```

```
temp_df = pd.DataFrame({'Algorithm':clfs.keys(), 'Accuracy_max_ft_3000':accuracy_scores, 'Precision_max_ft_3000':precision_scores}).sort_value
```

```
temp_df = pd.DataFrame({'Algorithm':clfs.keys(), 'Accuracy_scaling':accuracy_scores, 'Precision_scaling':precision_scores}).sort_values('Preci
```

```
new_df = performance_df.merge(temp_df,on='Algorithm')
```

```
new_df_scaled = new_df.merge(temp_df,on='Algorithm')
```

```
temp_df = pd.DataFrame({'Algorithm':clfs.keys(), 'Accuracy_num_chars':accuracy_scores, 'Precision_num_chars':precision_scores}).sort_values('F
```

```
new_df_scaled.merge(temp_df,on='Algorithm')
```

	Algorithm	Accuracy	Precision	Accuracy_scaling_x	Precision_scaling_x	Accuracy_scaling_y	Precision_scaling_y	Accuracy_num_chars
0	KN	0.905222	1.000000	0.905222	1.000000	0.905222	1.000000	0.905222
1	NB	0.970986	1.000000	0.970986	1.000000	0.970986	1.000000	0.970986
2	RF	0.973888	0.982609	0.973888	0.982609	0.973888	0.982609	0.973888
3	SVC	0.975822	0.974790	0.975822	0.974790	0.975822	0.974790	0.975822
4	ETC	0.974855	0.974576	0.974855	0.974576	0.974855	0.974576	0.974855
5	LR	0.955513	0.960000	0.955513	0.960000	0.955513	0.960000	0.955513
6	xgb	0.968085	0.948718	0.968085	0.948718	0.968085	0.948718	0.968085
7	GBDT	0.950677	0.930693	0.950677	0.930693	0.950677	0.930693	0.950677
8	BgC	0.958414	0.868217	0.958414	0.868217	0.958414	0.868217	0.958414
9	DT	0.930368	0.830000	0.930368	0.830000	0.930368	0.830000	0.930368